Collective-Aware Router Microarchitecture in gem5 Garnet

Multicast, express-link bypass, and tensor all-reduce

Chunyu Liu and Boyan Pu

September 7, 2026

Abstract

GPU training repeatedly maps gradient tensors and synchronization collectives onto the network. We study how preserving this communication structure in gem5 Garnet changes traffic, latency, and throughput. The implementation covers the two mechanisms required by Lab 4 Topic 3—physical bypass links and one-to-many multicast—and evaluates each against an appropriate baseline. On 4×4, 162 matched cases show 39.51% mean internal-link flit reduction, 3.638× geometric-mean latency speedup, and 190.89% mean throughput improvement relative to replicated unicast. On 8×8, the same fanouts show 34.95% traffic reduction, 3.472× latency speedup, and 208.34% throughput improvement across another 162 cases, with no throughput regressions. A further 108-case 8×8 scale-out study reaches 75.39% traffic reduction at fanout 64. The ten observed regressions are confined to low-fanout 4×4 cases and arise from atomic branch synchronization. The bypass design combines distance-scaled stride links, multi-hop dimension-order routing, and conservative congestion admission. Its 768 4×4 pairs achieve 1.291× geometric-mean latency speedup and +4.37% mean throughput; the corresponding 768 8×8 pairs achieve 1.583× and +39.37%. Their worst latency ratios are 0.909× and 0.954×, respectively. The plain-Mesh baseline has fewer links, ports, and buffers, so the comparison measures the performance of the augmented topology at its stated resource cost. A Topic 4 case study maps 15 scaled eight-H100 all-reduce events to multi-flit collective requests and shortens the simulated replay window by 4.728× relative to 6,720 serialized scalar-lane rounds, with identical source and Router-forwarded flit counts.

1 Introduction

Large-scale GPU training alternates local computation with global communication. During each iteration, GPU ranks produce gradient tensors, exchange them through collectives such as all-reduce and broadcast, and wait for the synchronized result before beginning the next step. As illustrated in Figure 1, this repeated dependency maps structured tensor operations onto packets and flits in the interconnection network. Network behavior therefore contributes directly to the synchronization path of training.

NCCL exposes the participant groups and operations associated with broadcast and all-reduce [7], but a conventional network reduces them to independent packets and destinations. The three mechanisms in this report preserve or exploit structure at complementary levels. Tree multicast shares common route prefixes for one-to-many delivery.

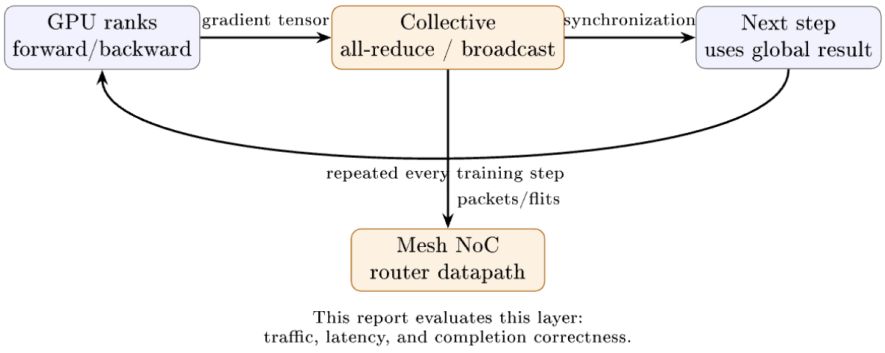


Figure 1: GPU training repeatedly maps gradient tensors and collective synchronization onto the Mesh NoC. This work studies how the Router datapath represents and transports that structure.

Tensor streaming retains an all-reduce event as a multi-flit collective request rather than serializing its lanes into thousands of independent rounds. Express links shorten selected paths through the underlying Mesh.

This organization also maps directly to the project specification. Lab 4 Topic 3 requires both Bypass and Multicast, together with appropriate baselines. We compare physical express-link bypass with plain Mesh XY and Router-level tree multicast with replicated unicast on the same Mesh. Topic 4 includes trace-based traffic, which we address by replaying measured H100 collectives through the tensor datapath. The evaluation asks:

1. Can Router-side replication reduce physical multicast traffic without changing the delivered destination set?
2. Do express links still improve latency on 4×4 and 8×8 after their span-dependent wire delay, added radix, and congestion are counted, and how does the result change with scale?
3. Can a traced all-reduce remain a multi-flit tensor request instead of thousands of serialized scalar-lane rounds?

Tree multicast shares common route prefixes and produces its largest traffic reduction at high fanout. Atomic fanout couples branch progress and can reduce throughput when one output is congested. The multi-hop, pressure-aware bypass primarily benefits loaded structured traffic. Event-level tensor packetization then carries the GPU-training narrative from group delivery to the representation and execution of complete all-reduce tensors.

Each result uses its corresponding unit: internal-link flit reduction, paired latency ratio, logical throughput change, or trace window. We report geometric means for latency ratios, arithmetic means for additive changes, medians and extrema for skewed distributions, and the number of regressions. Three deterministic seeds are retained as paired observations rather than collapsed before ratio calculation.

Section 2 introduces the shared Garnet substrate and metrics. Sections 3 and 4 present multicast and bypass, respectively, each from design and implementation through validation and evaluation. Section 5 presents the trace-driven H100 tensor study. The final sections discuss cross-mechanism scope, authorship, and reproducibility.

2 Background and common methodology

2.1 Relation to prior work

Garnet provides the detailed cycle-level network substrate used here [2]. Virtual Circuit Tree Multicasting established Router-level tree replication as an alternative to repeated unicast [3]; our implementation instead uses a compact destination bitmap and atomic branch allocation. Express Virtual Channels bypass intermediate Router pipelines over conventional channels [4]; our mechanism instead installs explicit span-2 physical links and therefore pays added port, buffer, and wire proxies. For routing safety, the offline oracle applies the channel-dependency condition of Dally and Seitz [5] to its generated static paths. SHARP demonstrates hardware streaming aggregation over reduction trees [6]; the tensor extension applies the same broad principle to lane-tagged Garnet packets and evaluates the effect of preserving event-level request structure.

The implementation therefore covers both Topic 3 mechanisms and adds one Topic 4 trace-based extension. The following definitions specify the baseline and measurement procedure for each comparison.

2.2 Common experimental methodology

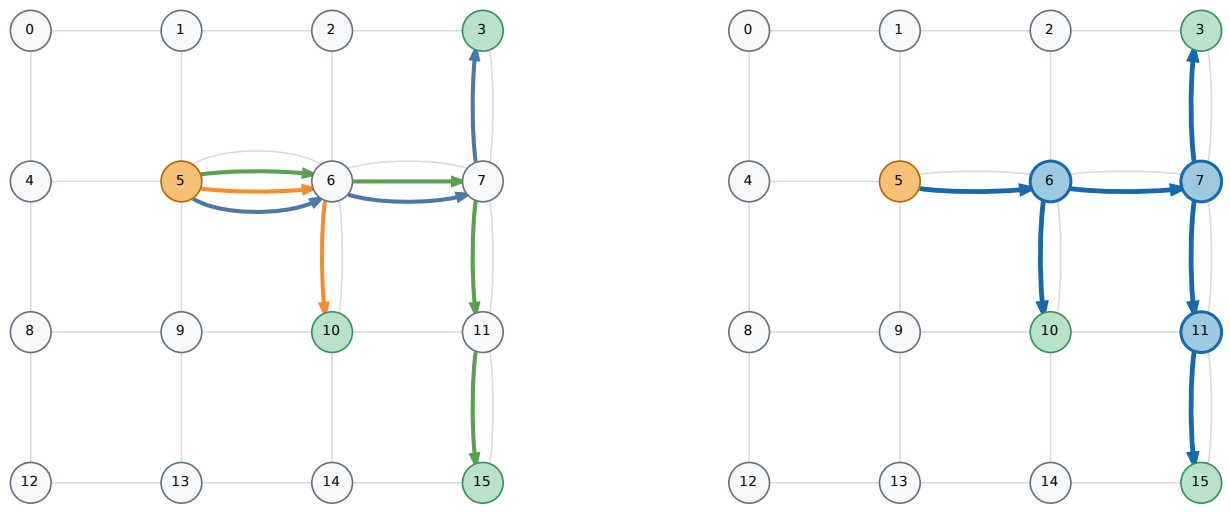
The implementation is based on gem5 v23.0.0.1 [1] and the cycle-level Garnet model [2]. Plain Mesh and collective trees use deterministic X-first XY routing. Evaluated revisions and source hashes are listed in Section 7.

Each performance comparison is paired: baseline and proposed runs use the same topology, workload realization, packet size, load, and seed. Multicast uses replicated unicast as its baseline, bypass uses plain Mesh XY, and the tensor study compares scalar-lane and event-level representations of the same trace. Mechanism-specific configurations and validation are stated in their own sections rather than combined into one global parameter table.

For a paired latency L, internal-link flit count F, and throughput Q, we use

$$S_L = L_{\text{base}}/L_{\text{new}}, \qquad R_F = 1 - F_{\text{new}}/F_{\text{base}}, \qquad I_Q = Q_{\text{new}}/Q_{\text{base}} - 1.$$

Thus $S_L > 1$ is faster, $R_F > 0$ is less link traffic, and $I_Q < 0$ is a throughput regression. One multicast request completes only after every selected destination receives the complete packet; Q counts these logical requests per cycle, so both variants perform the same application-level work. For bypass, Q is completed flits per node per source cycle. We use arithmetic means for additive changes and geometric means for latency ratios. The latter is essential for bypass because near-saturation ratios have a long positive tail. The analysis data retain every multicast pair together with aggregate bypass medians, extrema, and regression counts. Full result-set locations are documented in Section 7.



(a) Replicated-unicast baseline **(b) Proposed tree multicast**

Figure 2: Illustrative 4×4 multicast baseline and proposed path.

3 Tree multicast

3.1 Design and baseline

The baseline Network Interface injects one physical packet per remote destination and records a selected local destination directly. The multicast path injects one packet carrying a 64-bit destination bitmap. Each Router computes the required pruned-tree child branches from that bitmap and makes a local copy only where selected. Head, body, and tail state is retained per branch. Fanout is atomic: a flit advances only when all selected output VCs have credit. This preserves exact multi-flit delivery, but it also couples a fast branch to a blocked branch—an important interpretation of the throughput results below. The bitmap bounds one collective domain to 64 Routers.

Figure 2 contrasts replicated unicast with the shared-prefix tree used by the proposed mechanism.

3.2 Garnet implementation

The head flit first computes the pruned-tree children selected by its bitmap. The Router commits branch state only if every required output has a free VC; body and tail flits reuse those VCs. Listing 1 is an excerpt from the implemented two-phase allocation. This makes a multi-flit packet exact and prevents partial branch allocation, but also explains the measured head-of-line coupling: one blocked branch stalls the whole fanout.

Listing 1: Source excerpt: atomic multicast branch allocation (`Router.cc`).

```
const int vnet = head_flit->get_vnet();
for (const int destination : destinations) {
    const int outport = collectiveOutportForChild(destination);
    if (outport < 0 || !m_output_unit[outport]->has_free_vc(vnet)) {
        m_network_ptr->recordMulticastCreditStall();
        return false;
    }
}
for (const int destination : destinations) {
    const int outport = collectiveOutportForChild(destination);
    const int outvc = m_output_unit[outport]->select_free_vc(vnet);
    fatal_if(outvc < 0, "Router %d lost multicast VC allocation", m_id);
    m_multicast_branches.push_back({outport, outvc, destination});
}
```

3.3 Experimental setup and validation

For every multicast pair, the baseline and tree runs use the same Mesh, destination set, 1/4/16-flit packet size, request probability 0.1/0.5/1.0, optional uniform background load 0/0.1, and seed 1, 7, or 17. The balanced matrix uses source Router 5 on 4×4 and Router 27 on 8×8 with fanouts 4, 8, and 16; the separate 8×8 scale-out adds fanouts 32 and 64. Each run uses 20 warm-up, 100 measured, and 20 cooldown requests. Destination sets below full fanout are sampled without replacement and may include the source Router.

Correctness validation covers payload values, delivery sets, flit ordering, and completion under backpressure. The functional multicast matrix contains 30 destination/topology cases from 2×2 through 8×8, two implementations, and four packet sizes: 240 executions, or 120 mode-matched comparisons. The performance study contains 324 balanced cross-topology pairs and 108 additional 8×8 scale-out pairs. Analysis scripts check pair completeness, finite metrics, delivery metadata, and run counts before generating aggregate figures. All 240 executions match the destination bitmap and flit order without missing or duplicate delivery.

3.4 Results on a 4-by-4 Mesh

Tree multicast reduces duplicated physical-link traversal, while its throughput depends on synchronization among output branches. We therefore report link traffic and logical-request throughput separately.

The 4×4 study contains 162 matched cases at fanouts 4, 8, and 16. Tree multicast reduces internal-link flits by 39.51% on average (median 41.18%, range 16.67–53.13%) and achieves 3.638× geometric-mean latency speedup (median 3.588×, range 1.200–22.512×). Mean throughput improves by 190.89%, with a median improvement of 121.66%.

Ten 4×4 cases regress in throughput, all at fanout 4; the worst change is −18.24%. These cases occur when longer packets and higher load make one blocked output hold the other tree branches. The tree still performs less link work, but atomic fanout couples its progress to the slowest branch.

3.5 Results on an 8-by-8 Mesh

The matched 8×8 study first repeats the same fanouts 4, 8, and 16 across 162 cases. It reduces internal-link flits by 34.95% on average (median 29.63%, range 6.67–52.94%) and achieves 3.472× geometric-mean latency speedup (median 2.667×, range 1.161–20.163×). Mean throughput improves by 208.34%, with a median improvement of 145.39%. None of these 162 cases regresses; the smallest throughput change is +0.25%.

The 8×8 scale-out study adds 108 cases at fanouts 32 and 64. Mean traffic reduction is 62.41% at fanout 32 and 75.39% at fanout 64. Their geometric-mean latency speedups are 11.25× and 21.62×, while mean throughput improves by 895.54% and 1,888.29%, respectively. The larger groups expose more shared route prefixes, so Router-side replication removes an increasing fraction of repeated link traversal. Figure 3 shows the two topologies without merging their results.

3.6 Cross-topology multicast comparison

For the common fanouts, 4×4 saves 4.56 percentage points more link traffic, whereas 8×8 has the larger mean throughput improvement and no negative cases. Latency speedup is similar: 3.638× on 4×4 and 3.472× on 8×8. These per-topology results are the primary comparison; the fanout-32/64 cases characterize only 8×8 scale-out and are not pooled into the cross-topology means. Latency includes both shared-prefix replication and the dedicated Router datapath, while internal-link flit count isolates the traffic effect. The ten negative cases reported above provide the distribution boundary relevant to this comparison.

3.7 Multicast sensitivity and tradeoff

On 4×4, mean throughput changes at fanouts 4, 8, and 16 are +34.38%, +154.87%, and +383.42%, respectively. By packet size, the changes are +261.65% (1 flit), +169.45% (4 flits), and +141.57% (16 flits). The fanout-4 group contains all ten regressions; longer packets and higher load hold branch reservations for more cycles.

On 8×8, the corresponding common-fanout throughput changes are +44.57%, +165.45%, and +415.01%. By packet size, they are +276.71%, +188.30%, and +160.01%. Every subgroup remains nonnegative. The separate scale-out groups increase mean throughput by 895.54% at fanout 32 and 1,888.29% at fanout 64. Figure 4 presents the latency sensitivity in separate topology panels; throughput is stated directly because its topology and negative-case summaries are already given above.

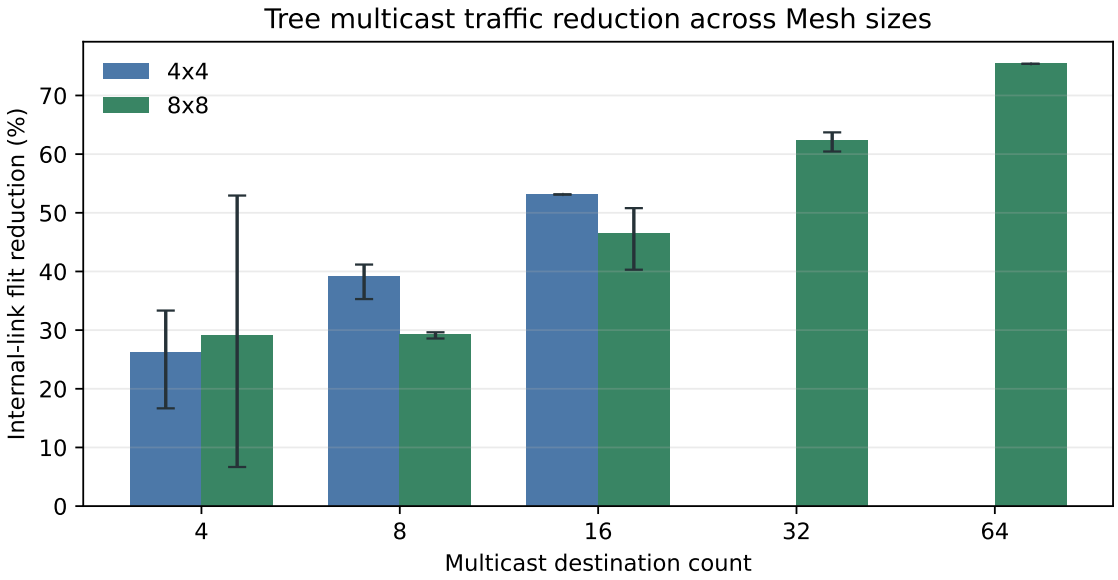


Figure 3: Internal-link flit reduction reported separately for 4×4 and 8×8 meshes. Bars are means and whiskers show observed ranges; fanouts 32 and 64 belong only to the 8×8 scale-out study.

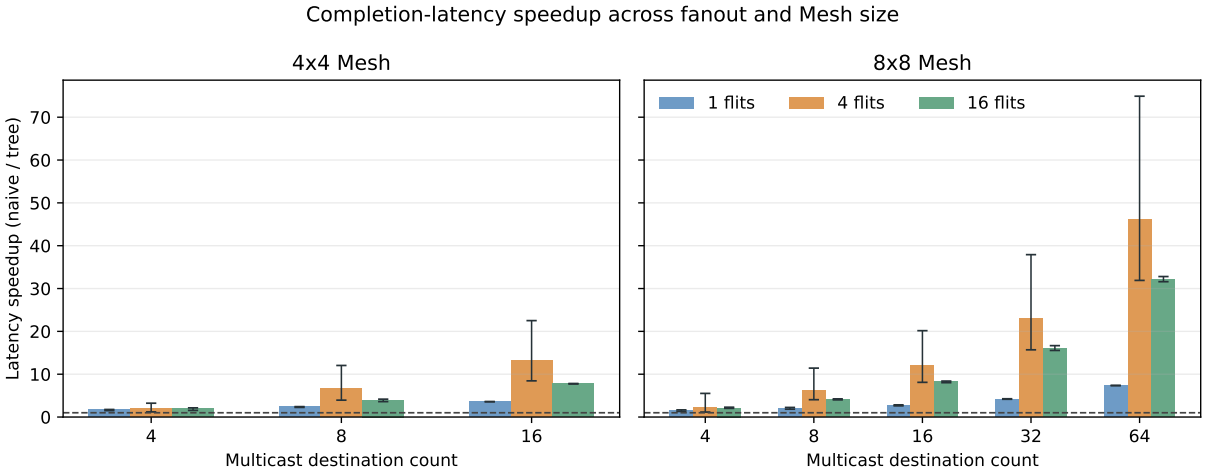


Figure 4: Multicast latency speedup by topology, fanout, and packet size; whiskers show observed ranges.

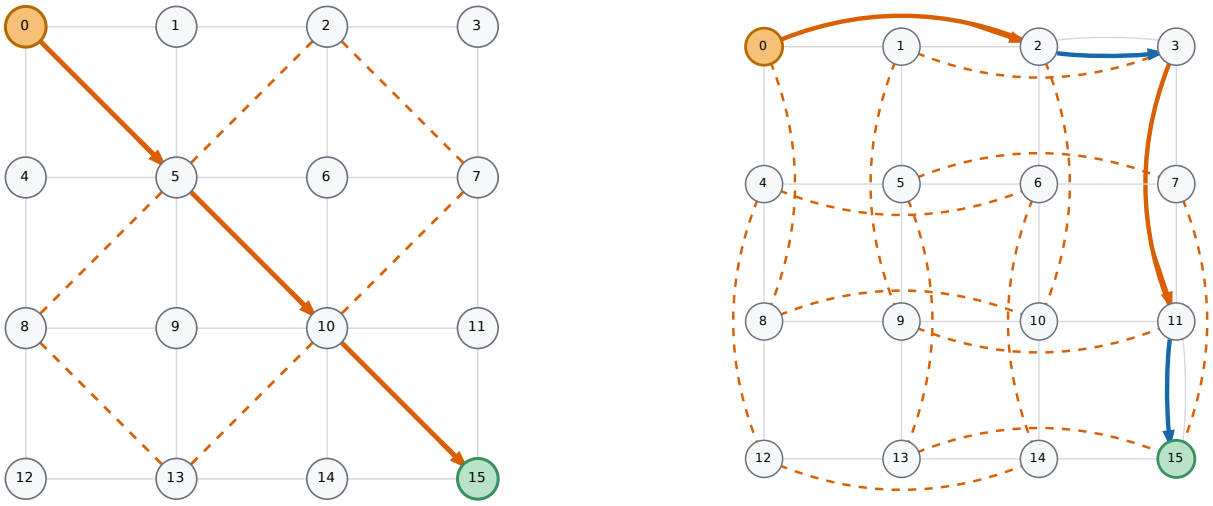
4 Pressure-aware express-link bypass

4.1 Proposed topology and baseline

The proposed `Mesh_Bypass` design appends bidirectional stride-2 links to the ordinary Mesh. Before simulation, a route-table builder enumerates every current-Router/destination pair. It may select dimension-aligned stride links repeatedly in the X phase and then the Y phase, but only when an express edge lowers the modeled cycle cost of the complete remaining path. The builder materializes all routes and rejects the placement if their channel-dependency graph is cyclic.

The selected design uses a distance-scaled wire model: each express-link latency scales with its Manhattan span. An ablation also evaluated an optimistic model in which every express link costs one cycle; that model is treated only as an upper bound on the value of shortening a path. The plain Mesh XY comparator has fewer links, ports, VCs, and buffers, so this is a workload-matched, added-resource comparison rather than an iso-area, iso-power, or iso-wire claim.

The stride topology is the primary design. Figure 5 contrasts it with an alternative phase-ordered diagonal placement.



(a) Phase-ordered diagonal alternative

(b) Proposed stride-2 topology

Figure 5: Illustrative 4×4 bypass placements and selected multi-hop routes. The diagonal alternative uses a phase-ordered X*→diagonal*→Y* chain; the primary stride design repeatedly selects aligned express links while preserving DOR.

Legend: orange dashed lines are installed bidirectional bypass links; solid orange arrows are selected express hops; solid blue is the ordinary DOR segment; gray lines are ordinary Mesh links.

4.2 Multi-hop routing and runtime admission

Topology construction creates two directed Garnet links for every installed physical shortcut and scales their latency by Manhattan span. The cost of an ordinary edge is link latency plus Router latency. An express edge uses its configured wire latency plus one Router traversal and is admitted only if it lowers the complete suffix cost. Listing 2 shows the dynamic programming recurrence. Exact-cost ties prefer the ordinary edge.

Listing 2: Source excerpt: cycle-aware multi-hop DOR selection (`bypass_oracle.py`).

```
@lru_cache(maxsize=None)
def best_stride_step(current, destination):
    if current == destination:
        return 0, None
    ordinary_next = xy_step(current, destination)
    suffix_cost, _ = best_stride_step(ordinary_next, destination)
    choices = [(ordinary_edge_cost + suffix_cost, 0,
                ordinary_next, "", None)]
    for link in outgoing[current]:
        if not dor_express(link, destination):
            continue
        suffix_cost, _ = best_stride_step(link["dst"], destination)
        cost = int(link["latency"]) + router_latency + suffix_cost
        choices.append((cost, 1, link["dst"], link["name"], link))
    cost, _, _, _, selected = min(choices)
    return cost, selected
```

Ordered Vnets retain the deterministic table. For unordered traffic, the selector may fall back to monotonic XY when the express or landing output is blocked, the express output is materially more congested, or the destination is a sustained hotspot. The destination-skew detector evaluates epochs of $4N$ data packets, enters the hot state at four times the uniform share, and retains it until the share falls below twice uniform. Packets longer than 32 flits additionally require credit headroom on the express and landing outputs.

Listing 3: Source excerpt: conservative runtime bypass admission (`RoutingUnit.cc`).

```
if (network->bypassDestinationSkewed(route.dest_router) ||
    express_free == 0 || escape_free == 0 ||
    (long_packet && lookahead_free == 0) ||
    express_free + 1 < xy_free)
```

```
    return xy_outport;

const int landing_capacity = vcs_per_vnet *
    network->getBuffersPerDataVC();
if (long_packet &&
    ((xy_free == vcs_per_vnet && xy_credits == landing_capacity) ||
     express_free < xy_free || express_credits < xy_credits ||
     lookahead_free < vcs_per_vnet ||
     lookahead_credits < landing_capacity))
    return xy_outport;
```

4.3 Routing safety and validation

The builder materializes every source–destination path, adds every successive channel pair to a dependency graph, and requires a topological ordering over the entire graph. A cycle formed across multiple routes therefore fails construction. Stride runtime adaptation can choose only the table's express edge or monotonic XY in the same DOR phase; both strictly approach the destination, complete X before Y, and never reverse dimension. The diagonal variant separately audits its full adaptive choice union under $X^* \rightarrow \text{diagonal}^* \rightarrow Y^*$ phase ordering.

Route/dependency, traversal, and multi-flit/backpressure suites pass 5, 8, and 14 cases, respectively. The proposed-stride performance inventory contains 3,072 runs, or 1,536 matched pairs. Analysis checks pair completeness, finite metrics, route metadata, and run counts before aggregation.

4.4 Experimental setup

Each pair uses the same traffic pattern, packet size, offered load, seed, and pre-generated source/destination counts on plain Mesh XY and Mesh_Bypass. The 4×4 and 8×8 matrices use uniform random, transpose, bit complement, and 50% hotspot traffic; 1/4/16/64-flit packets; 16 offered-flit points from 0.01 to 3.0 per node/source cycle; and seeds 1, 7, and 17. Each run uses 1,000 warm-up, 100,000 drain, 5,000 measurement, and 1,000,000 cooldown source cycles, giving 768 matched pairs per Mesh size. The main evaluation uses data VNet 2 with four buffers per VC and distance-scaled wires.

4.5 Results on a 4-by-4 Mesh

The main bypass experiment evaluates the distance-scaled stride-2 design against plain Mesh XY using four synthetic traffic patterns. In `uniform_random`, each source chooses a destination uniformly at random. In `transpose`, a source at mesh coordinate (x, y) sends to (y, x), creating a structured permutation. In `bit_complement`, it sends to the mirrored coordinate $(W-1-x, H-1-y)$, pairing opposite locations in the mesh. In `hotspot`, 50% of packets target one fixed Router (Router $N/2$ in an N-node mesh) and the rest use uniform random destinations. These patterns isolate route structure and contention. Each pattern is evaluated under the packet sizes, loads, and seeds defined above.

The policy consults the cycle-aware table at every Router, so a packet may use several stride links while remaining monotonic in X and then Y. For unordered data traffic, conservative admission retains the static shortcut unless the express or landing output is blocked, the express output is materially more congested than XY, or epoch counters identify the destination as a sustained hotspot. For packets above 32 flits, the selector also checks credit headroom and the oracle-selected output at the landing Router, admitting the shortcut only when XY is under pressure and the landing output is idle. A 96-pair pilot rejected a more aggressive version despite its higher mean because it introduced four regressions larger than 5%; the retained guard had none in the same pilot.

On 4×4, the evaluated design achieves 1.291× geometric-mean latency speedup and a 1.068× median. Mean throughput improves by 4.37%, while Router traversals per delivered flit fall by 17.37%. Of the 768 ratios, 117 are below one, but only one regresses by more than 5%. The worst ratio is 0.909× for 16-flit hotspot traffic at offered load 0.3. The smaller Mesh offers fewer long paths over which a stride link can amortize its added wire latency, so its gain is primarily a controlled demonstration that multi-hop admission can retain a net benefit without assuming one-cycle express wires.

4.6 Results on an 8-by-8 Mesh

The 8×8 experiment repeats the same traffic patterns, packet sizes, loads, and seeds for another 768 matched pairs. Geometric-mean latency speedup rises to 1.583×, with a 1.132× median. Mean throughput improves by 39.37%, and Router traversals per delivered flit fall by 18.84%. Although 126 individual latency ratios are below one, none

regresses by more than 5%; the worst ratio is 0.954×. Longer routes create more opportunities for repeated stride hops, while conservative admission avoids turning those opportunities into a large negative tail.

4.7 Cross-topology bypass comparison

Table 1 keeps the two Mesh sizes separate. The 8×8 Mesh has substantially larger latency and throughput gains, while the Router-traversal reduction changes by only 1.47 percentage points. The scaling benefit therefore reflects not only skipped Routers but also greater relief of loaded Router stages on longer paths. The 1.430× geometric mean over all 1,536 pairs remains an inventory summary rather than the primary topology result.

Table 1: Topology-separated performance of the distance-scaled, multi-hop stride design.

Mesh	Pairs	Speedup	Median	Throughput	Router/flit red.	$> 5\%$ regress.
4×4	768	1.291×	1.068×	+4.37%	17.37%	1
8×8	768	1.583×	1.132×	+39.37%	18.84%	0

These gains use added hardware rather than an iso-resource baseline. Relative to plain Mesh, the 4×4 and 8×8 stride topologies add 16 and 96 undirected links, wire-span sums of 32 and 192, maximum network degrees of 6 and 8 excluding the local NI port, and buffer-slot proxies of 768 and 4,608. The latter counts directed express input ports times four VCs times the configured control- and data-VNet slots; physical area and power remain outside Garnet.

5 Trace-driven H100 tensor all-reduce (Topic 4 extension)

The synthetic experiments above isolate network mechanisms; this trace-based Topic 4 study reconnects them to the GPU training loop in Figure 1. It preserves the ordering and tensor sizes of measured collectives while comparing scalar-lane and event-level request representations. In this way, the case study extends the multicast question from one-to-many delivery to the reduce-and-broadcast structure of all-reduce.

5.1 Tensor datapath implementation

Tensor packets retain their multi-flit boundary while each flit carries a lane identifier. A collective flit is intercepted before ordinary input-VC insertion, route computation, switch allocation, and crossbar traversal. The Router collective state machine accumulates each (collective, lane) key and forwards the completed value through a queued reduce/broadcast path. Completed lanes wait when the next output VC is unavailable, so backpressure does not discard reduction state.

Listing 4: Abridged source excerpt: lane-wise tensor accumulation (`Router.cc`).

```
const int lane = t_flit->get_lane_id();
const auto key = std::make_pair(t_flit->get_collective_id(), lane);
CollectiveLaneState& state = m_collective_lanes[key];
state.sum += t_flit->get_value();
++state.count;
if (state.count == m_collective_expected_fanin) {
    const int64_t sum = state.sum;
    m_collective_lanes.erase(key);
    if (m_collective_root)
        forwardCollectiveLane(sum, lane, CollectiveOp::Broadcast,
                              m_id, t_flit);
    else
        forwardCollectiveLane(sum, lane, CollectiveOp::Reduce,
                              parent, t_flit);
}
```

5.2 Trace lowering and experimental setup

The input comes from an eight-H100 80GB HBM3 NCCL microbenchmark (PyTorch 2.8.0+cu128, CUDA 12.8, NCCL 2.27.3). The replay compiler selects 15 measurement-phase broadcasts and 15 all-reduces: five repetitions each at 1, 4, and 16 MiB. It represents communication with 16-byte flits and scales payload bytes and relative release times by 1/1024. An all-reduce event is therefore 1, 4, or 16 KiB in the replay, corresponding to 64, 256, or 1,024 flits per rank and 6,720 lanes per rank across all 15 events. Scaling preserves event ordering and produces a tractable Garnet replay; the reported times are simulation ticks rather than native H100 or NVLink timings.

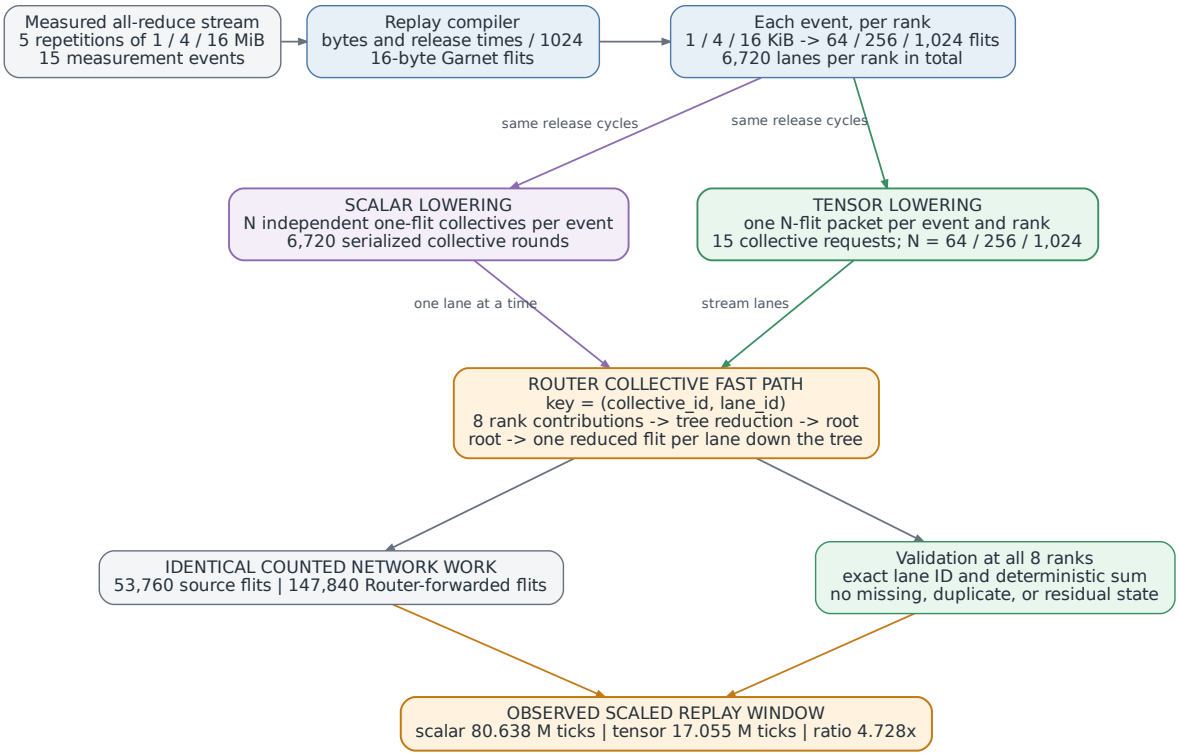


Figure 6: Scalar and tensor lowering of the same scaled all-reduce events. The manipulated variable is packet/request representation; release cycles, lane arithmetic, routes, and counted network work are held fixed.

For broadcast, the compiler splits events into variable-length tree-multicast packets. For all-reduce, scalar lowering repeats each event's release cycle for N independent one-flit collective rounds. Tensor lowering instead creates one N-flit packet per event and rank, with a lane ID on every flit. Both execute the same lane-wise Router reduction followed by tree broadcast. The JSON records chunks of at most 4 KiB after scaling. The Garnet loader combines these chunks into each event's total `tensor_flits` and executes one logical tensor request per event. The scalar and tensor paths are distinct from bitmap-based `tree_multicast`, although all three use shared tree prefixes.

Figure 6 connects the exact trace sizes, the two lowerings, the Router operation, and the two controlled outcomes.

The replay runs on Mesh XY and on a four-link, distance-scaled diagonal bypass topology. Every selected route in this 2×4 placement follows ordinary XY, so the second topology checks that unused links leave the replay unchanged.

5.3 Correctness and backpressure validation

Fourteen functional cases verify exact lane counts and reduced values across varied meshes, and 32 backpressure cases vary link latency, Router latency, and occupancy while requiring completion without residual accumulator state. Twelve trace-tool unit tests check schema, event sizes and order, collective IDs, and participants. In the evaluated replay, both all-reduce representations complete the same 53,760 rank-lane deliveries and 147,840 Router-forwarded flits; every lane ID and deterministic sum matches.

5.4 Replay experiments and results

The broadcast compiler expands 15 source events into 30 tree-multicast packets because each scaled 16-KiB event is represented by four 4-KiB packets. Both topology arms complete 6,720 source flits and 210 request-destination deliveries in a 17,047,500-tick window. The all-reduce comparison holds the route and scaled release schedule fixed while changing scalar-lane versus event-level packetization.

Table 2 records the three replay representations. Both topologies produce identical results.

Table 2: Scaled H100 replay summary. Logical requests are collective-level operations; source flits are network injections summed across participating ranks.

Replay view	Trace events	Logical requests	Source flits	Window (ticks)
Broadcast	15	30	6,720	17,047,500
Scalar all-reduce	15	6,720	53,760	80,638,000
Tensor all-reduce	15	15	53,760	17,054,500

All accumulator entries are released at completion. Event-level tensor packetization reduces the replay window in Table 2 from 80,638,000 to 17,054,500 ticks, a 4.728× improvement on both topologies. Since the two representations inject and forward the same number of flits, the difference comes from replacing 6,720 independently admitted scalar-lane rounds with 15 event-level collective requests. The experiment therefore measures the scheduling effect of request representation in the collective datapath.

6 Division of labor

Chunyu Liu: Garnet collective plumbing, tree multicast, express-link bypass design and evaluation, H100 trace capture and compilation, aggregate analysis, figures, and report integration.
Boyan Pu: Tensor all-reduce implementation, replay tooling, backpressure and correctness validation, tensor experiments and statistics, and presentation slide-deck (PPT) production.

7 Reproducibility

The evaluated multicast/tensor implementation is revision `77fcf26d57`. The multi-hop bypass evaluation records source revision `1e8764cde7`, with source hashes matching the pressure-aware implementation at `ea4c3c9b0f`. Machine-readable manifests record software versions, exact arguments, measurement windows, source hashes, and raw-result locations. The analysis scripts verify counts, finite metrics, pair completeness, and VNet/routing metadata before producing summaries.

Multicast and architecture figures can be rebuilt from committed inputs with:

```
python3 report/scripts/plot_multicast.py
python3 report/scripts/generate_architecture_figures.py
```

The validation runner `run_lab4_full_gate.py` executes seven functional, backpressure, and trace suites. The proposed bypass campaign is reproduced separately from the full functional gate.

8 Conclusion

The implementation satisfies both microarchitecture requirements in Lab 4 Topic 3: physical links bypass intermediate Routers, and Router-side replication sends one packet to multiple destinations. Each mechanism is evaluated against an explicit baseline under matched workloads. On 4×4, tree multicast removes 39.51% of internal-link flits and achieves 3.638× geometric-mean latency speedup across 162 matched cases; its ten throughput regressions all occur at fanout 4. On 8×8, the common-fanout matrix removes 34.95% of link traffic, achieves 3.472× latency speedup, and has no throughput regressions across another 162 cases. The separate 8×8 scale-out study reaches 75.39% traffic reduction at fanout 64. This separation identifies atomic branch synchronization as a 4×4 low-fanout effect rather than a general multicast trend. The latency results reflect both shared-prefix replication and the dedicated Router datapath. The bypass design reconsults a cycle-cost oracle at every Router and admits distance-scaled stride hops under conservative local and landing pressure. On 4×4 it achieves 1.291× geometric-mean latency speedup and +4.37% mean throughput; on 8×8 the corresponding results rise to 1.583× and +39.37%. The worst ratios are 0.909× and 0.954×, respectively. Mesh_Bypass adds links, ports, and buffers, making this an added-resource comparison with explicit topology-specific cost proxies. The H100 trace then extends the collective study to complete tensors: preserving 15 event-level multi-flit all-reduce requests shortens the scaled replay window by 4.728× with identical counted network work.

Together, the experiments show how exposing communication structure to the network removes duplicated transport and unnecessary request scheduling. Tree replication shares multicast prefixes, event-level tensor packets avoid

scalar-lane serialization, and pressure-aware routing directs express traffic according to modeled cycle cost and current congestion.

References

[1] N. Binkert et al., "The gem5 Simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, 2011, pp. 1–7. doi: 10.1145/2024716.2024718.

[2] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator," *ISPASS*, 2009, pp. 33–42. doi: 10.1109/ISPASS.2009.4919636.

[3] N. E. Jerger, L.-S. Peh, and M. H. Lipasti, "Virtual Circuit Tree Multicasting: A Case for On-Chip Hardware Multicast Support," *ISCA*, 2008, pp. 229–240. doi: 10.1109/ISCA.2008.12.

[4] A. Kumar, L.-S. Peh, P. Kundu, and N. K. Jha, "Express Virtual Channels: Towards the Ideal Interconnection Fabric," *ISCA*, 2007, pp. 150–161. doi: 10.1145/1250662.1250681.

[5] W. J. Dally and C. L. Seitz, "Deadlock-Free Message Routing in Multiprocessor Interconnection Networks," *IEEE Transactions on Computers*, vol. 36, no. 5, 1987, pp. 547–553. doi: 10.1109/TC.1987.1676939.

[6] R. L. Graham et al., "Scalable Hierarchical Aggregation and Reduction Protocol (SHARP) Streaming-Aggregation Hardware Design and Evaluation," *High Performance Computing*, 2020. https://pmc.ncbi.nlm.nih.gov/articles/PMC7295336/.

[7] NVIDIA, "NVIDIA Collective Communication Library Documentation." https://docs.nvidia.com/deeplearning/nccl/user-guide/.